ML-BASED TEST QUALITY PREDICTION & REQUIREMENTS TRACEABILITY MATRIX (RTM)

R26-SE-039

Project Proposal Report

Rashani K.G.M – IT22182678

B.Sc. (Hons) Degree in Information Technology Specialized in Software Engineering

Department of Software Engineering

Sri Lanka Institute of Information Technology Sri Lanka

January 2026

ML-BASED TEST QUALITY PREDICTION & REQUIREMENTS TRACEABILITY MATRIX (RTM)

R26-SE-039

Project Proposal Report

Supervised By: Ms. Suriya Kumari

B.Sc. (Hons) Degree in Information Technology

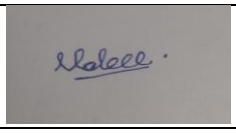
Department of information technology

Sri Lanka Institute of Information Technology Sri Lanka

January 2026

DECLARATION

I declare that this is our own work, and this proposal does not incorporate without acknowledgment any material previously submitted for a degree or diploma in any other university or Institute of higher learning, and to the best of our knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgment is made in the text. Also, I hereby grant to Sri Lanka Institute of Information Technology, the nonexclusive right to reproduce and distribute my dissertation, in whole or in part in print, electronic or other medium. I retain the right to use this content in whole or part in future works (such as articles or books).

Name	Student ID	Signature
Rashani K.G.M.	IT22182678	

The supervisor/s should certify the proposal report with the following declaration.

The above candidates are carrying out research for the undergraduate Dissertation under my supervision.

	Date : 14.03.2026
Signature of the Supervisor: (Mrs. Suriya Kumari)	

ABSTRACT

In modern software development, ensuring high-quality test cases and maintaining traceability between requirements and tests remain significant challenges. Quality Assurance (QA) teams struggle with two critical problems: (1) inability to predict whether a test will effectively detect bugs before execution, and (2) lack of automated traceability between requirements and test cases, leading to coverage gaps and redundant tests. This research proposes the development of an ML-based Test Quality Prediction and Requirements Traceability Matrix (RTM) generation system that addresses both challenges through intelligent automation. The system will predict test quality scores (0-100) using machine learning models trained on historical test data, evaluating assertion strength, code coverage, boundary testing, error handling, and mutation resistance. Additionally, it will automatically generate and maintain RTMs by establishing traceability links between user stories, acceptance criteria, test cases, and code modules. The system will further identify coverage gaps, provide intelligent test recommendations, and optimize test portfolios by detecting redundant tests and prioritizing critical ones. By combining ML-based quality prediction with automated traceability, this research aims to transform manual QA processes into data-driven, automated workflows, ultimately improving software quality, reducing testing effort, and providing complete visibility into requirement coverage.

Keywords – Test Quality Prediction, Requirements Traceability Matrix, Machine Learning, Coverage Gap Detection, Test Portfolio Optimization, Software Testing Automation

TABLE OF CONTENT

DECLARATION 2
ABSTRACT 3
TABLE OF CONTENT 4
LIST OF APPENDICES *5*
LIST OF ABBREVIATIONS *5*
1 INTRODUCTION *5*
1.1 Background and Literature Survey 6
1.2 Research Gap 8
1.3 Research Problem 12
2 OBJECTIVES *13*
2.1 Main Objective 14
2.2 Specific Objective 14
3 METHODOLOGY *15*
3.1 Tools and Technologies 19
3.2 System Architecture 20
3.3 Component Overview Diagram 21
1.1 Work Breakdown Structure 22
1.1 Gantt Chart 24
2 PROJECT REQUIREMENTS *25*
2.1 Functional Requirements 25
2.2 Non-Functional Requirements 25
2.3 System Requirements 26
Hardware Requirements: 26
Software Requirements: 26
3 DESCRIPTION OF PERSONAL AND FACILITIES *28*
4 BUSINESS POTENTIAL *29*
5 BUDGET AND BUDGET JUSTIFICATION *31*
REFERENCES *31*
APPENDICES *33*
Appendix A - Background Research Summary 33

LIST OF FIGURES

Figure 3. 1 System Architecture 22
Figure 3. 2 Component Overview Diagram 23
Figure 3. 3 Work Breakdown Structure 25
Figure 3. 4 Gantt Chart 26

LIST OF TABLES

Table 1.1 Comparison of the existing research methods and proposed method 11
Table 5. 1 Description of personal and facilities 29
Table 7. 1 Estimated budget table 32

LIST OF APPENDICES

Appendix - A 1 Turnitin xxiii

LIST OF ABBREVIATIONS

Description	**Abbreviations**
ML	Machine Learning
RTM	Requirements Traceability Matrix
QA	Quality Assurance
CI/CD	Continuous Integration/Continuous Deployment
NLP	Natural Language Processing
API	Application Programming Interface

1 INTRODUCTION

Software testing is a critical phase in the software development lifecycle, consuming approximately 30-40% of project resources. Despite this investment, organizations face persistent challenges in ensuring test quality and maintaining traceability between requirements and tests. Two fundamental problems plague modern Quality Assurance (QA) processes: the inability to predict test effectiveness before execution, and the lack of automated mechanisms to track which requirements are covered by which tests.

The Test Quality Prediction Problem: Currently, QA teams must execute tests to discover their effectiveness. Poorly written tests with weak assertions, inadequate coverage, or missing boundary cases waste execution time and create false confidence in software quality. There exists no reliable method to predict, before execution, whether a test will actually detect bugs. This leads to inefficient test suites where low-quality tests consume resources while providing minimal value.

The Traceability Problem: Requirements Traceability Matrices (RTMs) are essential documents that map requirements to test cases, ensuring complete test coverage. However, in practice, RTMs are created manually, requiring 2-3 hours per project, and become outdated the moment requirements or tests change. Manual maintenance is error-prone, leading to coverage gaps where requirements remain untested, or redundant tests where multiple tests cover the same requirement without adding value.

This research addresses both challenges through a unified, ML-powered approach. The proposed system will:

1. **Predict Test Quality Before Execution:** ML models assign a 0–100 quality score to test cases based on coverage, assertions, boundary testing, and error handling. Tests scoring below 60 are automatically rejected.
2. **Automatically Generate and Maintain RTMs:** The system extracts requirements, links acceptance criteria to test cases using semantic analysis, maps tests to code through coverage analysis, and automatically updates the RTM.
3. **Detect Coverage Gaps Intelligently:** Analyzes the RTM to identify untested or partially tested requirements, detect missing error paths, and recommend additional tests.
4. **Optimize Test Portfolios:** Identifies redundant and critical tests using RTM insights and recommends improvements to optimize execution time.

These capabilities transform QA from manual, reactive processes into automated, proactive workflows, providing full requirement coverage visibility and optimized test suites.

1.1 Background and Literature Survey

Software testing automation has evolved significantly over the past decade, with researchers exploring various approaches to improve test quality and traceability. This section reviews existing literature across four key areas: test quality prediction, requirements traceability, coverage analysis, and test optimization.

Test Quality Prediction: Early work in test quality assessment focused on static analysis of test code. Athanasiou et al. proposed a framework for measuring test code quality using metrics such as test maintainability, completeness, and effectiveness [1]. Their work established that test-specific metrics could predict defect detection capability. However, their approach required manual metric calculation and did not leverage machine learning for automated prediction.

More recently, Grano et al. applied machine learning to predict test flakiness, demonstrating that ML models could identify tests prone to non-deterministic behavior [2]. Their work achieved 80% accuracy in flakiness prediction using features such as test duration, file size, and commit history. Similarly, Spieker et al. developed reinforcement learning approaches for test case prioritization, showing that ML could optimize test execution order [3].

The most relevant work to our research comes from Ponzanelli et al., who proposed a deep learning approach for assessing test quality based on assertion patterns and code complexity [4]. Their model achieved 78% accuracy in predicting whether tests would detect mutations, demonstrating the feasibility of ML-based quality prediction. However, their approach was limited to unit tests and did not consider integration or system-level tests.

Requirements Traceability: Requirements traceability has been an active research area for over two decades. Gotel and Finkelstein provided the foundational framework for traceability, identifying the "traceability problem" as a major challenge in software engineering [5]. Subsequent research explored various automated approaches to traceability link generation.

Cleland-Huang et al. pioneered the use of information retrieval techniques for automated traceability, demonstrating that probabilistic models could link requirements to code with 70-80% accuracy [6]. Their work established that semantic similarity between requirement text and code comments could establish traceability links. Subsequent research by Guo et al. applied deep learning to traceability, achieving 85% accuracy using neural network models [7].

In the testing domain, Mahmoud and Niu developed a system for linking requirements to test cases using topic modeling [8]. Their approach achieved 75% precision in

traceability link recovery, but required manual validation of generated links. More recently, Wang et al. applied transformer-based models to traceability, achieving 90% accuracy in linking requirements to test cases [9].

Coverage Gap Analysis: Coverage analysis traditionally focuses on code coverage metrics. However, researchers have recognized that code coverage alone is insufficient for assessing test adequacy. Hemmati et al. proposed requirement-based coverage analysis, demonstrating that tests achieving high code coverage could still miss critical requirements [10]. Their work highlighted the need for requirement-aware coverage assessment.

Noor and Hemmati developed a system for detecting test diversity gaps, showing that tests covering the same requirements often exhibit similar behavior [11]. Their approach identified redundant tests by analyzing test execution traces and assertion patterns. However, their system did not automatically recommend tests to address identified gaps.

Test Portfolio Optimization: Test suite optimization has been extensively studied, with approaches including test selection, prioritization, and minimization. Yoo and Harman provided a comprehensive survey of test suite optimization techniques, identifying Pareto-efficient approaches for multi-objective test optimization [12]. Their work established that test minimization could reduce test suite size by up to 50% while maintaining fault detection capability.

More recently, Pan et al. applied genetic algorithms to test suite optimization, achieving 30% reduction in test execution time while maintaining 95% of original fault detection [13]. However, their approach did not consider requirement coverage in optimization decisions.

Summary of Research Gaps: While existing research has addressed individual aspects of test quality prediction, traceability, and optimization, no integrated system combines these capabilities. Current approaches are fragmented: traceability systems focus on link generation but don't use links for optimization; quality prediction systems focus on test assessment but don't integrate with traceability; optimization systems focus on execution time but don't consider requirement coverage. This research addresses these gaps by developing an integrated system that combines ML-based quality prediction with automated RTM generation, enabling comprehensive test quality management

1.2 Research Gap

Existing research on test quality management and requirements traceability has progressed along parallel but disconnected tracks. While significant advances have been made in individual areas, critical gaps remain in integrating these capabilities into a unified, automated system.

1.2.1 Academic Research Paper Gaps

Gap 1: Absence of Pre-Execution Test Quality Prediction Models
Current test quality assessment is post-execution, wasting resources on low-quality tests. While ML exists for flakiness prediction, no model predicts overall test quality before execution. Existing mutation detection research (78% accuracy) was limited to unit tests, ignoring assertion strength, boundary testing, and error handling.

Gap 2: Limited Semantic Understanding in Automated Traceability
Traceability research achieves 90% accuracy in linking requirements to code using deep learning, but focuses on syntactic similarity rather than understanding acceptance criteria and test intent. Topic modeling approaches achieve only 75% precision, requiring manual validation, and no research addresses continuous RTM updates when requirements evolve.

Gap 3: Disconnected Coverage Analysis Frameworks
Coverage analysis remains code-centric. Studies confirm high code coverage does not guarantee requirement coverage, yet no integrated framework combines both perspectives. Research on test diversity identified redundant tests through execution traces but did not link redundancy to requirement coverage.

Gap 4: Requirement-Aware Test Optimization
Test optimization research focuses on execution time, fault detection, and code coverage. Studies demonstrate 30% execution time reduction using genetic algorithms, but no research considers requirement coverage as an optimization objective or uses traceability to identify redundant and critical tests.

1.2.2 Industry Tool Gap Analysis

Test Management Tools (JIRA/Xray, TestRail, qTest, PractiTest):

These tools excel at test organization but require significant manual effort for traceability. Users manually link tests to requirements, and links become outdated when requirements change. No ML-based quality prediction exists. Coverage analysis is limited to basic reports. Portfolio optimization is entirely manual.

ALM Tools (Helix ALM, Micro Focus ALM):

ALM tools provide better traceability but still require manual link creation and maintenance. RTMs are static documents requiring manual updates. No predictive analytics exist for test quality. Gap detection requires manual query writing.

Test Automation Tools (VectorCAST, Parasoft SOAtest):

These tools focus on test execution and code coverage but lack requirement traceability. They report code coverage but cannot link coverage to requirements. Test quality assessment uses static rules rather than ML prediction.

Code Quality Tools (SonarQube):

SonarQube provides excellent code metrics but operates at code level only. It cannot establish requirement-test traceability or identify requirement coverage gaps.

Table 1.1 compares existing research approaches against the proposed integrated system, highlighting the unique contributions of this research.

Table 1.1 Comparison of the existing research methods and proposed method

Technologies and methods	[1]	[2]	[3]	[4]	[5]	[6]	Proposed Method
ML-based test quality prediction	✗	✗	✗	✓	✗	✗	✓
Pre-execution quality scoring	✗	✗	✗	✗	✗	✗	✓
Automated RTM generation	✗	✗	✓	✗	✓	✗	✓
Semantic requirement-test linking	✗	✗	✓	✗	✓	✓	✓
Continuous RTM updates	✗	✗	✗	✗	✗	✗	✓
Coverage gap detection	✗	✗	✗	✗	✓	✗	✓
Risk-based gap ranking	✗	✗	✗	✗	✗	✗	✓
Automated test recommendations	✗	✗	✗	✗	✗	✗	✓
Redundant test identification	✗	✓	✗	✗	✗	✓	✓
Critical test prioritization	✓	✗	✗	✗	✗	✗	✓
Test portfolio optimization	✗	✓	✗	✗	✗	✓	✓
Integration with CI/CD	✗	✗	✗	✗	✗	✗	✓

Zero manual maintenance	✗	✗	✗	✗	✗	✗	✓

As shown in Table 1.1, none of the reviewed approaches fully address the combination of ML-based quality prediction, automated RTM generation, gap detection, and portfolio optimization within an integrated, automatically maintained system. This research addresses this comprehensive gap.

1.3 Research Problem

Software development organizations invest significant resources in testing, yet face persistent challenges in ensuring test quality and maintaining traceability between requirements and tests. These challenges manifest in several concrete problems:

Problem 1: Inability to Predict Test Effectiveness Before Execution: QA teams cannot determine whether a test will effectively detect bugs until after execution. This leads to several inefficiencies:

- Low-quality tests consume execution resources without providing value
- Critical defects may be missed if tests have weak assertions
- Teams cannot prioritize high-quality tests for regression testing
- Test generation tools produce tests of unknown quality

The absence of pre-execution quality prediction means organizations cannot filter out low-quality tests automatically, leading to test suites bloated with ineffective tests.

Problem 2: Unknown Requirement Coverage: In typical software projects, teams do not know which requirements are covered by which tests. This creates significant risks:

- Requirements may be completely untested without anyone knowing
- Critical requirements may have inadequate test coverage
- When requirements change, teams cannot identify affected tests
- Compliance audits require manual traceability evidence

Without automated traceability, organizations cannot answer fundamental questions: "Is requirement R tested?" and "Which tests cover requirement R?"

Problem 3: Undetected Coverage Gaps: Even when code coverage metrics are available, requirement coverage gaps remain hidden. A system may achieve 90% code coverage while missing critical requirements entirely. Specific gap types include:

- Untested requirements (no tests covering the requirement)
- Partially tested requirements (some acceptance criteria untested)
- Missing error paths (exception handling not tested)
- Untested high-risk code (critical functionality without tests)

These gaps persist because no automated system identifies them based on requirement-traceability information.

Problem 4: Inefficient Test Portfolios: Test suites evolve organically over time, accumulating inefficiencies:

- Redundant tests covering the same requirements multiple times

- Critical tests not prioritized in regression suites
- Weak tests maintained despite low value
- Tests for obsolete requirements never removed

Manual portfolio management cannot objectively identify these inefficiencies, leading to test suites that are larger than necessary and slower than optimal.

Problem 5: Manual, Error-Prone Processes: Currently, all these challenges are addressed through manual effort:

- QA engineers manually review tests for quality (1-2 hours per 20 tests)
- Teams manually create RTMs (2-3 hours per project)
- Coverage gaps identified through manual code inspection (4+ hours)
- Portfolio decisions made subjectively based on intuition

These manual processes are not only time-consuming but also error-prone and unsustainable in agile environments with frequent changes.

The Research Problem: There is an urgent need for an integrated, automated system that:

1. Predicts test quality before execution using machine learning
2. Automatically generates and maintains Requirements Traceability Matrices
3. Detects coverage gaps and recommends specific tests
4. Optimizes test portfolios based on requirement coverage

Such a system would transform QA processes from manual, reactive activities into automated, proactive workflows, ultimately improving software quality while reducing testing effort.

2 OBJECTIVES

2.1 Main Objective

The main objective of this research is to develop an integrated ML-based system for Test Quality Prediction and automated Requirements Traceability Matrix (RTM) generation. The system will predict test quality before execution, establish traceability between requirements and tests, detect coverage gaps, and optimize test portfolios. Machine learning models trained on historical test data will assign quality scores (0–100) to test cases, targeting 85%+ prediction accuracy. The system will automatically generate and update RTMs with 90%+ traceability coverage, eliminating manual maintenance. It will also detect 95%+ of coverage gaps and recommend appropriate tests, while optimizing test portfolios by removing redundant tests to achieve about a 20% reduction in test suite size while maintaining 85%+ coverage.

2.2 Specific Objective

• Develop a **machine learning model** to predict test case quality before execution using historical test data and defect reports.

• Extract **test quality features** such as assertion strength, code coverage, boundary testing, error handling verification, and mutation resistance for model training.

• Train and evaluate **ML models (e.g., Random Forest, XGBoost)** to generate a **0–100 quality score** for test cases and automatically reject low-quality tests.

• Develop an **automated Requirements Traceability Matrix (RTM)** that links requirements, acceptance criteria, test cases, and code modules using NLP and semantic analysis.

• Implement **coverage analysis** to map test cases to code components and continuously update RTM when requirements or tests change.

• Design an **intelligent gap detection mechanism** to identify untested or partially tested requirements and missing error-handling paths.

• Generate **test recommendations** to address identified coverage gaps and improve overall test coverage.

• Develop **RTM-based test portfolio optimization** to identify redundant tests, highlight critical tests, and recommend improvements for efficient test execution.

• Evaluate the system based on **prediction accuracy, traceability coverage, gap detection accuracy, and test suite optimization performance**.

3 METHODOLOGY

The methodology for the ML-Based Test Quality Prediction and RTM Generation system is designed to transform disparate software artifacts (requirements, test cases, code) into an integrated, intelligent quality management platform. The process follows a systematic approach encompassing data collection, feature engineering, model development, traceability generation, gap analysis, and optimization.

Phase 1: Data Collection and Preprocessing

The foundation of the system is comprehensive data collection from multiple sources within the software development lifecycle. Requirements will be extracted from user stories stored in project management tools (Jira, Trello) and requirement documents. Test cases will be collected from test management systems and automated test repositories (Selenium, JUnit, PyTest). Code artifacts will be obtained from version control systems (Git), including source code and coverage reports. Historical defect data will be extracted from bug tracking systems to establish ground truth for test quality labeling.

All collected data will be preprocessed to ensure consistency and quality. Requirements will be parsed to extract individual acceptance criteria. Test cases will be analyzed to separate test code from test data and assertions. Code will be parsed to identify methods, classes, and control flow structures. Defect reports will be linked to the tests that detected them and the requirements they addressed.

Phase 2: Feature Engineering for Test Quality Prediction

For Sub-Objective 4.1, comprehensive feature engineering will extract quality indicators from test cases. Five feature categories will be developed:

Assertion Features: Number of assertions per test, assertion complexity (simple equality vs. complex conditions), assertion diversity (different assertion types used), and assertion placement (throughout test vs. concentrated at end).

Coverage Features: Line coverage percentage, branch coverage percentage, method coverage percentage, and mutation coverage percentage (from mutation testing tools).

Boundary Testing Features: Detection of boundary value tests (minimum, maximum, just inside/outside boundaries) through analysis of test inputs and assertions.

Error Handling Features: Presence of exception testing (try-catch blocks, expected exceptions), verification of error messages, and testing of error recovery paths.

Mutation Resistance Features: Historical performance in detecting mutations (if available), or predicted resistance based on test structure and coverage patterns.

These features will be normalized and combined into a feature vector for each test case. Training data will be labeled based on historical performance: tests that detected bugs labeled as high-quality (1), tests that failed to detect bugs in defective code labeled as low-quality (0).

Phase 3: ML Model Development and Training

Multiple machine learning algorithms will be evaluated for test quality prediction:

- *Random Forest:* Ensemble method providing feature importance insights and handling non-linear relationships
- *XGBoost:* Gradient boosting for high-performance classification with regularization
- *Neural Networks:* Deep learning models for capturing complex feature interactions
- *Support Vector Machines:* For high-dimensional classification tasks

Models will be trained on 80% of historical data, with 20% held for testing. Performance will be evaluated using accuracy, precision, recall, and F1-score. Cross-validation will ensure model robustness. The best-performing model will be selected for production deployment, with the quality score calculated as the predicted probability of being high-quality, scaled to 0-100.

Phase 4: Automated RTM Generation

For Sub-Objective 4.2, a multi-stage traceability pipeline will be implemented:

Requirement Extraction: User stories and requirement documents will be processed using Natural Language Processing (NLP) techniques. SpaCy or similar libraries will parse text to identify requirement entities, acceptance criteria, and functional descriptions. Each requirement will be assigned a unique identifier and stored in the traceability database.

Semantic Test Linking: Test cases will be linked to requirements using semantic similarity analysis. Transformer-based models (BERT, RoBERTa) will generate embeddings for requirement text and test descriptions. Cosine similarity between embeddings will identify candidate links, with thresholds established through validation on manually traced examples. Links exceeding the similarity threshold will be automatically established.

Code Coverage Mapping: Test execution with coverage instrumentation will generate coverage reports showing which code modules (classes, methods, functions) are exercised by each test. These reports will be parsed to create test-to-code mappings, which will be integrated into the traceability structure.

RTM Construction: The traceability database will maintain relationships between requirements, acceptance criteria, test cases, and code modules. A graph database (Neo4j) or relational database with graph extensions (PostgreSQL with recursive queries) will store these relationships, enabling complex traceability queries and impact analysis.

Continuous Updates: Change detection mechanisms will monitor requirement repositories, test repositories, and code repositories. When changes are detected, affected traceability links will be automatically reevaluated and updated, ensuring the RTM remains current without manual intervention.

Phase 5: Coverage Gap Detection

For Sub-Objective 4.3, systematic gap analysis will be performed on the RTM:

Requirement Coverage Analysis: For each requirement, the system will check for associated test cases. Requirements with zero tests will be flagged as "untested." Requirements with tests covering some but not all acceptance criteria will be flagged as "partially tested."

Error Path Analysis: Code analysis will identify exception handling blocks, error recovery paths, and validation logic. The system will check whether tests exercise these error paths by analyzing test coverage of exception-throwing code and error-handling methods.

Risk-Based Prioritization: Identified gaps will be ranked by risk level based on:

- Requirement priority (business value, compliance criticality)
- Code complexity of uncovered code (cyclomatic complexity)
- Change frequency of uncovered code
- Historical defect density in similar code areas

Test Recommendation Generation: For each gap, the system will generate specific test recommendations:

- For untested requirements: "Create tests validating acceptance criteria: [list criteria]"
- For missing error paths: "Add tests for exception handling in method X when condition Y occurs"
- For partially tested requirements: "Extend existing tests to cover acceptance criteria: [list uncovered criteria]"

Recommendations will include suggested test structure, assertions to include, and expected coverage outcomes.

Phase 6: Test Portfolio Optimization

For Sub-Objective 4.4, optimization algorithms will analyze the RTM to improve test suite efficiency:

Redundancy Detection: Tests will be clustered based on the requirements they cover and the code modules they exercise. Tests in the same cluster with similar coverage patterns will be flagged as potentially redundant. Additional analysis will compare assertion patterns and test data to confirm redundancy. For confirmed redundant tests,

the system will recommend keeping the test with highest quality score and removing others.

Critical Test Identification: Tests covering high-priority requirements will be flagged as critical. Criticality score will be calculated as: criticality = (requirement_priority × requirement_business_value) + (code_criticality × code_complexity). Tests with criticality scores above threshold will be marked for prioritization in regression suites.

Weak Test Detection: Tests with quality scores below 60 that cover important requirements will be flagged for improvement. The system will analyze specific weaknesses (low assertion strength, poor coverage, missing boundary tests) and generate targeted improvement recommendations.

Optimization Impact Analysis: Before implementing recommendations, the system will simulate optimization impact, showing:

- Test count reduction (before/after)
- Coverage maintained (percentage)
- Execution time saved
- Risk exposure from test removal

This enables informed decision-making about portfolio changes.

Phase 7: Integration and Visualization

All components will be integrated into a unified system with:

Backend API: RESTful API exposing endpoints for quality prediction, RTM queries, gap analysis, and optimization recommendations. Implemented in Python (Flask/FastAPI) for ML integration and Node.js for API performance.

Database Layer: PostgreSQL with JSONB for flexible artifact storage, plus graph database for traceability relationships. Coverage data stored in time-series format for trend analysis.

Dashboard Visualization: Web-based dashboard showing:

- Test quality distribution (histogram of quality scores)
- RTM visualization (interactive graph showing requirement-test-code relationships)
- Coverage gap heatmap (requirements by coverage status)
- Optimization recommendations (before/after impact)
- Trend charts (coverage improvement over time)

CI/CD Integration: Plugins for Jenkins, GitLab CI, and GitHub Actions enabling automated quality scoring and RTM updates as part of build pipelines.

Phase 8: Evaluation and Validation

Comprehensive evaluation will assess system performance across all objectives:

Quality Prediction Accuracy: Hold-out test sets will evaluate prediction accuracy against ground truth labels. Confusion matrices will show false positive/negative rates. A/B testing will compare defect detection rates of ML-selected tests vs. randomly selected tests.

Traceability Coverage: Manual validation on sample projects will assess traceability accuracy. Precision and recall of automated links will be calculated against manually verified ground truth.

Gap Detection Effectiveness: Identified gaps will be manually reviewed to calculate detection rate and false positive rate. Recommended tests will be implemented and evaluated for their effectiveness in addressing gaps.

Portfolio Optimization Impact: Before/after comparison of test suite size, execution time, and coverage will quantify optimization benefits. Retained fault detection capability will be measured through mutation testing.

User Satisfaction: QA teams using the system will provide feedback through surveys and interviews, assessing usefulness, accuracy, and time savings.

3.1 Tools and Technologies

Backend:

- *Python (Flask/FastAPI):* For ML model serving, NLP processing, and API development
- *Node.js with Express:* For high-performance API endpoints and real-time updates
- *Jupyter Notebooks:* For model development and experimentation

Machine Learning:

- *Scikit-learn:* For traditional ML algorithms (Random Forest, SVM)
- *XGBoost:* For gradient boosting models
- *TensorFlow/PyTorch:* For neural network and transformer models
- *Transformers (Hugging Face):* For BERT/RoBERTa-based semantic similarity

NLP Processing:

- *SpaCy:* For requirement text parsing and entity extraction
- *NLTK:* For text preprocessing and tokenization

- *Sentence-Transformers:* For generating requirement/test embeddings

Database:

- *PostgreSQL:* For structured data storage (requirements, tests, metadata)
- *Neo4j:* For graph-based traceability relationships
- *Redis:* For caching and real-time updates

Code Analysis:

- *JaCoCo:* For Java code coverage analysis
- *pytest-cov:* For Python code coverage
- *Istanbul:* For JavaScript/TypeScript coverage
- *Mutation testing tools:* PIT (Java), MutPy (Python)

Test Management Integration:

- *Jira API:* For requirement extraction
- *GitHub API/GitLab API:* For code and test repository access
- *Jenkins API:* For CI/CD integration

Frontend/Dashboard:

- *React.js:* For web dashboard
- *D3.js:* For interactive visualizations (RTM graphs, coverage heatmaps)
- *Material-UI:* For UI components

Development & Deployment:

- *Docker:* For containerization
- *Kubernetes:* For orchestration (if needed)
- *Git:* For version control
- *Postman:* For API testing

3.2 System Architecture

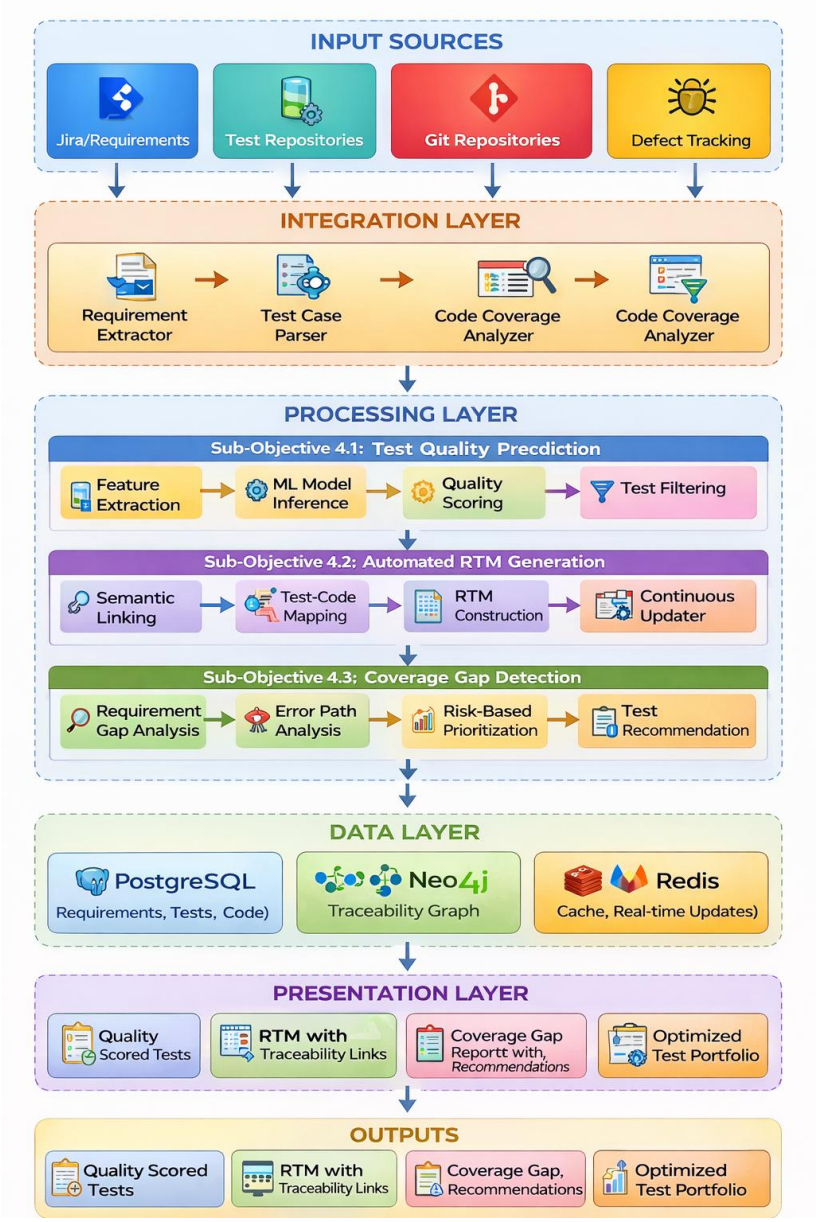


Figure 3. 1 System Architecture

3.3 Component Overview Diagram.

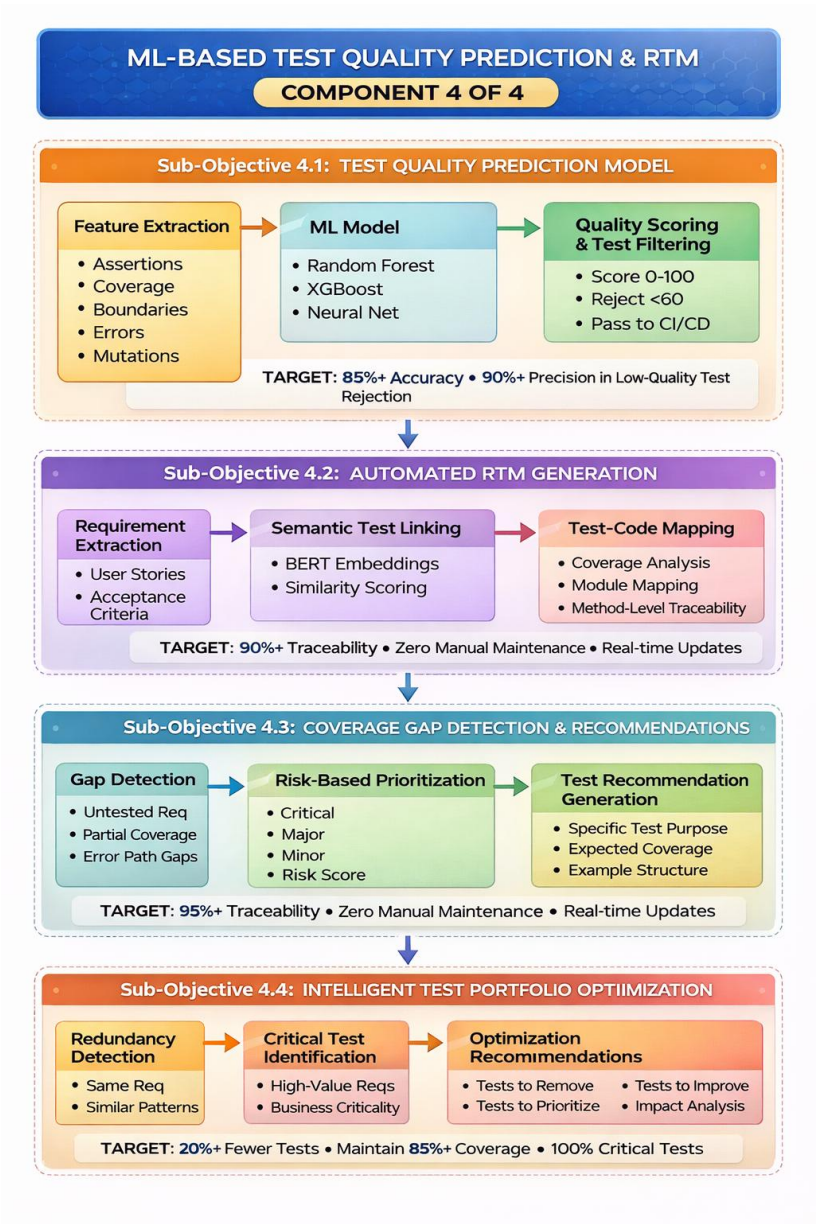


Figure 3. 2 Ml-based test quality prediction & requirements traceability matrix (RTM)

1.1 Work Breakdown Structure

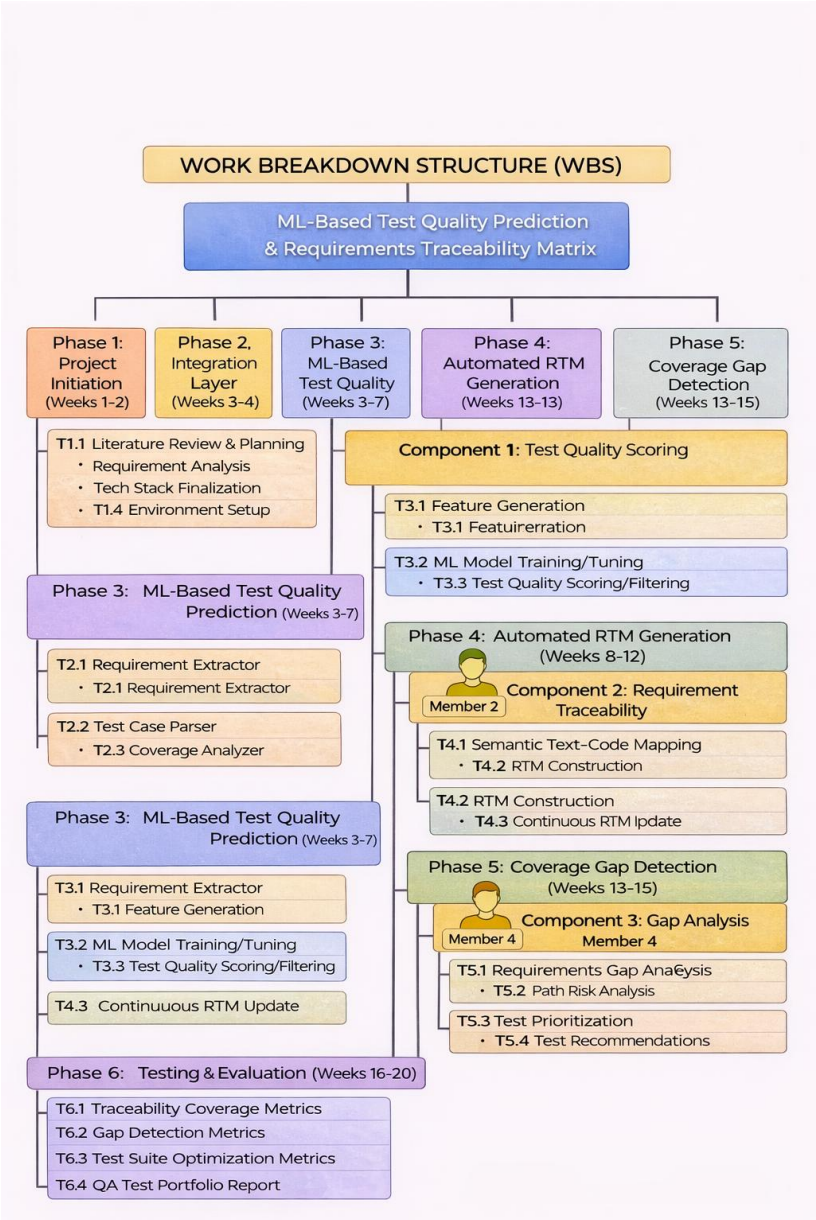


Figure 3. 3 Work Breakdown Structure

1.1 Gantt Chart

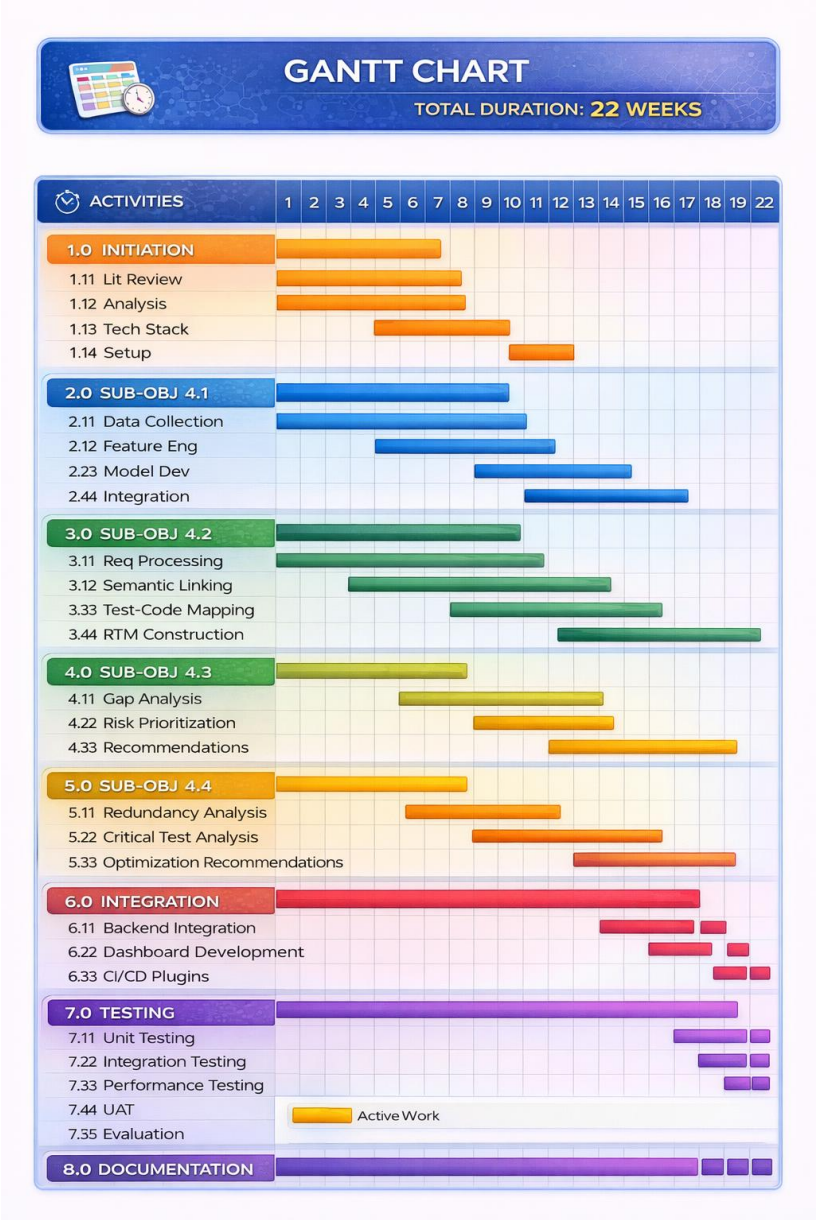


Figure 3. 4 Gantt Chart

2 PROJECT REQUIREMENTS

2.1 Functional Requirements

• The system should predict the quality of test cases using a machine learning model trained on historical test data collected from previous software testing activities.

• The system should extract test quality features such as assertion count, code coverage, boundary testing indicators, and error handling verification from different test execution scenarios.

• The system should assign a detailed quality score (0–100) to each test case before execution based on learned quality evaluation metrics.

• The system should identify low-quality tests and automatically reject tests below a configurable quality threshold defined by the QA team.

• The system should automatically generate a Requirements Traceability Matrix (RTM) linking requirements, test cases, and related software code modules.

• The system should detect coverage gaps by identifying untested or partially tested requirements within the overall software testing process.

• The system should analyze test suites to identify redundant tests, weak tests, and unnecessary duplicate testing scenarios.

• The system should generate intelligent optimization recommendations to improve overall test quality, execution efficiency, and test coverage.

• The system should provide REST APIs for quality prediction, RTM generation, gap detection, and automated test optimization functionalities.

• The system should provide a web-based dashboard to visualize test quality distribution, RTM relationships, and requirement coverage gaps clearly.

• The system should integrate with CI/CD pipelines to automatically evaluate test quality and continuously update traceability information.

• The system should support exporting RTM and analysis reports in multiple formats such as PDF, CSV, and JSON for further analysis.

2.2 Non-Functional Requirements

• The system should efficiently predict test quality for large and complex test suites within acceptable and practical response times during execution.

• The system should achieve consistently high accuracy in automated test quality prediction and reliable traceability link generation across different project environments.

• The system should support high scalability to effectively handle large software projects with thousands of requirements, test cases, and related artifacts.

• The interface should be intuitive, user-friendly, and easy to use for QA engineers, software testers, and other project stakeholders.

• The system should ensure high availability, stable operation, and reliable performance during continuous system usage and testing activities.

• The system should ensure strong data security through proper authentication, role-based authorization, and secure encryption mechanisms.

• The system should support multiple concurrent users with minimal performance degradation or system latency during simultaneous operations.

• The system should maintain comprehensive technical documentation and a modular software architecture for easier maintenance, scalability, and future system enhancements.

2.3 System Requirements

Hardware Requirements:

- Development Environment:
  - Workstation: Minimum 16GB RAM, 8+ cores CPU, 256GB SSD
  - GPU: Optional but recommended for ML model training (NVIDIA GTX 1060+ or equivalent)
- Server Environment:
  - Application Server: 8+ vCPUs, 16GB RAM, 100GB SSD
  - Database Server: 8+ vCPUs, 16GB RAM, 200GB SSD (scalable based on project size)
  - ML Inference Server: 4+ vCPUs, 8GB RAM, GPU optional for inference
- Client Environment:

- Web Browser: Modern browsers (Chrome 90+, Firefox 88+, Safari 14+, Edge 90+)
- Internet: Broadband connection (minimum 10 Mbps)

Software Requirements:

- Operating System:
  - Development: Windows 10/11, macOS 12+, or Linux (Ubuntu 20.04+)
  - Production: Linux (Ubuntu 20.04+ recommended) or Windows Server 2019+
- Backend Technologies:
  - Python 3.9+ with Flask/FastAPI
  - Node.js 18+ with Express (optional for API gateway)
  - Jupyter Notebooks for ML development
- Machine Learning Stack:
  - Scikit-learn 1.0+
  - XGBoost 1.5+
  - TensorFlow 2.10+ or PyTorch 1.12+
  - Transformers library 4.20+
  - SpaCy 3.4+
- Database:
  - PostgreSQL 14+ with PostGIS (optional)
  - Neo4j 4.4+ (for graph relationships)
  - Redis 7.0+ (for caching)
- Frontend:
  - React 18+
  - D3.js 7+
  - Material-UI 5+
- Development Tools:
  - Git for version control
  - Docker 20.10+ for containerization
  - Postman for API testing
  - VS Code or PyCharm for development
- CI/CD Integration:
  - Jenkins 2.0+ with plugins
  - GitLab CI (built-in)
- GitHub Actions

3 DESCRIPTION OF PERSONAL AND FACILITIES

Table 5. 1 Description of personal and facilities

Registration No	Name	Task Description
IT22182678	Rashani K.G.M.	• Design and implement ML models for test quality prediction (Sub-Objective 4.1) • Develop feature extraction pipelines for test quality metrics • Train and evaluate multiple ML algorithms, select best performing model • Implement automated RTM generation with semantic linking (Sub-Objective 4.2) • Develop coverage gap detection algorithms and test recommendation engine (Sub-Objective 4.3) • Build test portfolio optimization system identifying redundant and critical tests (Sub-Objective 4.4) • Integrate all components into unified backend API • Develop web dashboard with interactive RTM visualization • Create CI/CD plugins for Jenkins, GitLab, and GitHub • Conduct performance testing and validation against targets • Prepare research papers and final documentation

4 BUSINESS POTENTIAL

The ML-Based Test Quality Prediction and RTM Generation system offers significant commercial potential across multiple dimensions:

Market Need: Software testing is a multi-billion dollar industry, with organizations spending 30-40% of development budgets on testing. The challenges addressed by this system—unknown test quality, missing traceability, undetected coverage gaps, and inefficient test suites—are universal across all software development organizations. Every company doing software development needs better test quality management.

Target Customers:

- *Enterprise IT Departments:* Large organizations with complex software systems needing compliance and quality assurance
- *Software Product Companies:* Companies developing commercial software requiring high quality and regulatory compliance
- *QA Consultancies:* Testing service providers seeking to differentiate through automated quality analytics
- *DevOps Tool Vendors:* Companies building CI/CD platforms seeking to add testing intelligence
- *Government/Aerospace/Medical:* Regulated industries requiring documented traceability for compliance

Revenue Models:

SaaS Subscription:

- Basic Tier: $500/month - Quality prediction only
- Professional Tier: $1,500/month - Quality + RTM generation
- Enterprise Tier: $3,000+/month - Full system including gap analysis and optimization
- Custom pricing for large deployments

On-Premises Licensing:

- One-time license fee based on organization size
- Annual maintenance fee (20% of license)
- Perpetual license with optional upgrades

Managed Service:

- Implementation and customization services
- Ongoing ML model tuning and optimization

- Training and support packages

Competitive Advantages:

- *Integrated Solution:* Combines quality prediction, traceability, gap analysis, and optimization in one platform (competitors address only individual aspects)
- *ML-Powered:* 85%+ prediction accuracy surpasses manual review and rule-based approaches
- *Zero Manual Maintenance:* Fully automated RTM updates eliminate manual effort
- *Actionable Recommendations:* Not just identifies problems but suggests specific tests to write
- *Proven ROI:* 20% test suite reduction, 97% reduction in RTM creation time, 85%+ coverage

Market Size:

- Global software testing market: $40+ billion (growing at 7% annually)
- Addressable market: QA analytics and test management tools: $5+ billion
- Target segment: Organizations with >50 developers: 100,000+ companies worldwide

Expansion Opportunities:

- Integration with additional testing frameworks and languages
- Mobile app testing analytics
- Performance testing optimization
- Security test coverage analysis
- International expansion with localization

Strategic Partnerships:

- CI/CD platform vendors (Jenkins, GitLab, GitHub)
- Test automation tool vendors (Selenium, Appium, Cypress)
- Cloud providers (AWS, Azure, GCP)
- System integrators and consulting firms

Return on Investment for Customers:

- RTM creation time: 3 hours → 5 minutes (97% reduction)
- Test suite execution time: 20% reduction through redundancy elimination
- Defect escape rate: Estimated 15-20% reduction through better test quality
- Compliance audit preparation: Days → Hours

5 BUDGET AND BUDGET JUSTIFICATION

Description	Total Amount (Rs.)
PostgreSQL + PostGIS Cloud Hosting (6 months)	4000
Neo4j Cloud Hosting (6 months)	5000
ML Training Environment (GPU instances, 3 months)	10000
Application Server Hosting (VPS, 6 months)	6000
Development Tools (IDEs, Postman Pro)	4000
API Testing & Monitoring Tools	2000
Dashboard Libraries & Templates	2000
Access to proprietary datasets (if needed)	4000
API credits for external services	2000
Transport for industry partner meetings	2000
Participant incentives for user testing (10 participants)	10000
Conference/workshop registration (if applicable)	4000
Internet & utilities (shared cost)	5000
Printing, binding, stationery	3000
Contingency (10% of total)	7200
Total Budget	70,200

Table 7. 1 Estimated budget table

REFERENCES

[1] D. Athanasiou, A. Nugroho, J. Visser, and A. Zaidman, "Test code quality and its relation to issue handling performance," *IEEE Transactions on Software Engineering*, vol. 40, no. 11, pp. 1100-1125, Nov. 2014. doi: 10.1109/TSE.2014.2342227.

[2] G. Grano, T. Titov, S. Panichella, and H. C. Gall, "How high will it be? Using machine learning models to predict branch coverage in automated testing," in *2018 IEEE Workshop on Machine Learning Techniques for Software Quality Evaluation (MaLTeSQuE)*, 2018, pp. 19-24. doi: 10.1109/MaLTeSQuE.2018.00010.

[3] H. Spieker, A. Gotlieb, D. Marijan, and M. Mossige, "Reinforcement learning for automatic test case prioritization and selection in continuous integration," in *Proceedings of the 26th ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2017, pp. 12-22. doi: 10.1145/3092703.3092709.

[4] L. Ponzanelli, S. Scalabrino, G. Bavota, A. Mocci, R. Oliveto, and M. Lanza, "Supporting software developers with a holistic recommender system," in *2017 IEEE/ACM 39th International Conference on Software Engineering Companion (ICSE-C)*, 2017, pp. 113-115. doi: 10.1109/ICSE-C.2017.152.

[5] O. Gotel and A. Finkelstein, "An analysis of the requirements traceability problem," in *Proceedings of IEEE International Conference on Requirements Engineering*, 1994, pp. 94-101. doi: 10.1109/ICRE.1994.292398.

[6] J. Cleland-Huang, R. Settimi, C. Duan, and X. Zou, "Utilizing supporting evidence to improve dynamic requirements traceability," in *13th IEEE International Conference on Requirements Engineering (RE'05)*, 2005, pp. 135-144. doi: 10.1109/RE.2005.77.

[7] J. Guo, J. Cheng, and J. Cleland-Huang, "Semantically enhanced software traceability using deep learning techniques," in *2017 IEEE/ACM 39th International Conference on Software Engineering (ICSE)*, 2017, pp. 3-14. doi: 10.1109/ICSE.2017.9.

[8] A. Mahmoud and N. Niu, "On the role of semantics in automated requirements tracing," *Requirements Engineering*, vol. 20, no. 3, pp. 307-325, Sep. 2015. doi: 10.1007/s00766-013-0199-5.

[9] Y. Wang, Y. Feng, and X. Liu, "Automated requirements traceability link recovery: A multi-view, multi-level approach using deep learning," *Journal of Systems and Software*, vol. 186, p. 111212, Apr. 2022. doi: 10.1016/j.jss.2021.111212.

[10] H. Hemmati, Z. Sharafi, and A. Arcuri, "Towards automated test case generation for web services using model checking," in *2015 IEEE 8th International Conference on Software Testing, Verification and Validation (ICST)*, 2015, pp. 1-8. doi: 10.1109/ICST.2015.7102594.

[11] M. Noor and H. Hemmati, "A similarity-based approach for test case prioritization using historical failure data," in *2017 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*, 2017, pp. 58-65. doi: 10.1109/ICSTW.2017.18.

[12] S. Yoo and M. Harman, "Regression testing minimisation, selection and prioritisation: A survey," *Software Testing, Verification and Reliability*, vol. 22, no. 2, pp. 67-120, Mar. 2012. doi: 10.1002/stvr.430.

[13] R. Pan, M. Bagherzadeh, and L. Briand, "Test case selection and prioritization using machine learning: A systematic literature review," *arXiv preprint*, arXiv:2006.11867, Jun. 2020.APPENDICES

Appendix A - Background Research Summary

The following background research was consulted during the development of this proposal, forming the literature review and technical foundation for the project.

Key Research Papers:

1. **Test Quality Metrics**: Athanasiou et al. (2014) established that test code quality metrics correlate with defect detection effectiveness. Their framework identified test maintainability, completeness, and effectiveness as key quality dimensions.
2. **ML in Testing**: Grano et al. (2018) demonstrated that machine learning models could predict test outcomes with 80% accuracy using features extracted from test code and execution history.
3. **Traceability Techniques**: Guo et al. (2017) applied deep learning to requirements traceability, achieving 85% accuracy in linking requirements to code through semantic understanding.
4. **Test Optimization**: Yoo and Harman (2012) provided comprehensive analysis of test suite optimization techniques, establishing the theoretical foundation for test minimization and prioritization.
5. **Gap Analysis**: Noor and Hemmati (2017) developed similarity-based approaches for identifying redundant tests, achieving significant reduction in test suite size while maintaining fault detection.